Elements of Statistical Learning Theory

Contents

8.1 Linear separability **1**

8.2 Convergence of Empirical Risk Minimization (ERM) **5**
8.2.1 The key questions of Statistical Learning Theory 5

8.3 The Vapnik-Chervonenkis (VC) dimension **8**

8.4 The growth function **10**
8.4.1 Defining the growth function 10
8.4.2 Revisit convergence of ERM 13

8.5 Results of statistical learning theory **15**

8.6 References **16**

8.1 Linear separability

Setting:

$$\left\{ \left(\underline{\mathbf{x}}^{(\alpha)}, y_{\mathrm{T}}^{(\alpha)} \right) \right\}, \quad \alpha = 1, \ldots, p$$

- p points
- N dimensions $\underline{\mathbf{x}} \in \mathbb{R}^N$
- binary assignment: $y_T \in \{-1, 1\}$

Model:

Linear neuron with N weights and a bias ($N + 1$ degrees of freedom).

Task:

Find the total number of binary assignments that are linearly separable.

1. Place p points in N dimensions, any way you like as long as they're not collinear.
2. Assign all possible labels to the points.

Example with 4 points in 2D:

Q 8.1.1: *How many assignment configurations are there?*

Q 8.1.2: *How many of them can be separated by a connectionist neuron?*

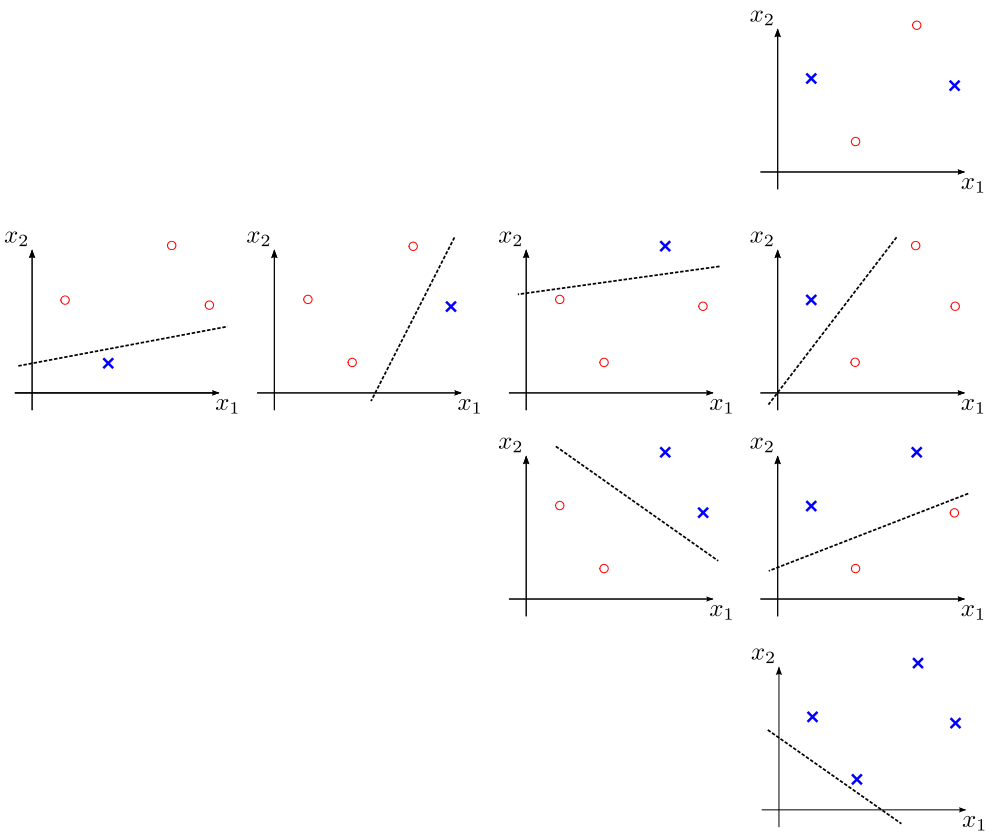


Figure 8.1.1: Classifying 4 points in 2D. Multiply by 2 (reverse classification) to get all possible $2^4 = 16$ configurations. 2 of those configurations, the XOR- (top-right) and XNOR-like configurations are not linearly separable.

Calculate the number of linearly separable assignments:

$$\tilde{C}_{(p,N)} := 2 \sum_{k=0}^{N} \underbrace{\binom{p-1}{k}}_{\substack{\text{Binomial} \\ \text{coefficient}}} \tag{8.1.1}$$

The Binomial coefficient[1] ("n choose k") is defined as:

$$\binom{n}{k} = \frac{n!}{k!(n-k)!} \tag{8.1.2}$$

for $n, k \in \mathbb{N}_0$ with $n \geq k \geq 0$

The Binomial coefficient represents the coefficient associated with the y^k term in the polynomial expansion $(1+y)^n$.

Consider the following example[2] of using polynomial expansion for calculating the fourth power of $(1+y)$:

$$(1+y)^4 = \binom{4}{0} y^0 + \binom{4}{1} y^1 + \binom{4}{2} y^2 + \binom{4}{3} y^3 + \binom{4}{4} y^4 \tag{8.1.3}$$

$$= 1 + 4y + 6y^2 + 4y^3 + y^4 \tag{8.1.4}$$

[1] See Pascal's triangle for a fun way on how to manually compute the coefficient.
[2] The example is taken from the Wikipedia article on Binomial coefficient

The binomial coefficient $\binom{4}{2} = \frac{4!}{2!(4-2)!} = 6$. 6 matches the coefficient in front of the y^2 term in the expansion in Eq.(8.1.4).

$$\tilde{C}_{(p,N)} := 2\sum_{k=0}^{N} \binom{p-1}{k}$$

Using our example with $p = 4$ and $N = 2$:

$$\begin{aligned}
\tilde{C}_{(4,2)} &= 2\sum_{k=0}^{2} \binom{4-1}{k} && (8.1.5)\\
&= 2\left[\binom{4-1}{0} + \binom{4-1}{1} + \binom{4-1}{2}\right] && (8.1.6)\\
&= 2\left[1+3+3\right] && (8.1.7)\\
&= 14 && (8.1.8)
\end{aligned}$$

But this looks slightly different from what is in the lecture. In the lecture it looked like this:

$$C_{(p,N)} := 2\sum_{k=0}^{N-1} \binom{p-1}{k} \tag{8.1.9}$$

Q 8.1.3: *Why does the sum for $\tilde{C}_{(p,N)}$ run to N instead of $N-1$?*

-The difference between $\tilde{C}_{(p,N)}$ from Eq.(7) and $C_{(p,N)}$ from Eq.(8.1.9) look different is due to the fact that $\tilde{C}_{(p,N)}$ computes the sum for $k = 1, \ldots, N$ instead of stopping at $N-1$. This way the sum iterates over the degrees of freedom in a connectionis neuron which has a bias term. Choosing a connectionist neuron to separate the classes involves $N+1$ degrees of freedom because we add the bias term (the ability to translate the hyperplane). Therefore, the reason betweenn the two definitions is the model choice and assuming that $C_{(p,N)}$ counts separability around the origin without the ability to shift the hyperplane.

-An alternative explanation to the difference between $\tilde{C}_{(p,N)}$ and $C_{(p,N)}$ is how to interpret N. The definition of $C_{(p,N)}$ treats N as the parameters of the hyperplane. Therefore the bias is implicit. If we assume that N is the number of components in $\underline{\mathbf{x}}$ and we need to prepend a bias term to it, we compute the number of separable configurations using $\tilde{C}_{(p,N)}$. It is important to understand that $\tilde{C}_{(p,N)}$ and $C_{(p,N)}$ effectively yield the same result when we feed each the appropriate arguments.

Recursive formula for the binomial coefficient:

$$\binom{n}{k} + \binom{n}{k-1} = \binom{n+1}{k} \tag{8.1.10}$$

Use the the recursive formula for the binomial coefficient to show that:

$$\tilde{C}_{(p,N)} + \tilde{C}_{(p,N-1)} = \tilde{C}_{(p+1,N)} \tag{8.1.11}$$

The relevance of Eq.(8.1.11) relationship becomes apparent when attempting to derive the VC dimension (introduced in Section 8.3) of a connectionist neuron.

$$\tilde{C}_{(p,N)} \quad + \quad \tilde{C}_{(p,N-1)} \quad = \tilde{C}_{(p+1,N)} \tag{8.1.12}$$

Plugging in Eq.(7) for each:

$$2\sum_{k=0}^{N}\binom{p-1}{k} + 2\sum_{k=0}^{N-1}\binom{p-1}{k} = 2\sum_{k=0}^{N}\binom{p+1-1}{k} \tag{8.1.13}$$

Simplify the LHS by modifying the sums such that they share the same limits.

$$2\binom{p-1}{0} + 2\sum_{k=\cancel{0}1}^{N}\binom{p-1}{k} + 2\sum_{k=0}^{N-1}\binom{p-1}{k} = 2\sum_{k=0}^{N}\binom{p}{k} \tag{8.1.14}$$

$$2\binom{p-1}{0} + 2\sum_{k=1}^{N}\binom{p-1}{k} + 2\sum_{k=1}^{N}\binom{p-1}{k-1} = \tag{8.1.15}$$

$$2\binom{p-1}{0} + 2\sum_{k=1}^{N}\Bigg[\underbrace{\binom{p-1}{k} + \binom{p-1}{k-1}}_{=\binom{p}{k}\text{ (using the recursive property)}}\Bigg] = \tag{8.1.16}$$

$$\underbrace{2\binom{p-1}{0}}_{=1=\binom{p}{0}} + 2\sum_{k=1}^{N}\binom{p}{k} = 2\sum_{k=0}^{N}\binom{p}{k} \tag{8.1.17}$$

$\binom{p}{0}$ can now be absorbed into the sum.

$$2\sum_{k=0}^{N}\binom{p}{k} = 2\sum_{k=0}^{N}\binom{p}{k} \tag{8.1.18}$$

8.2 Convergence of Empirical Risk Minimization (ERM)

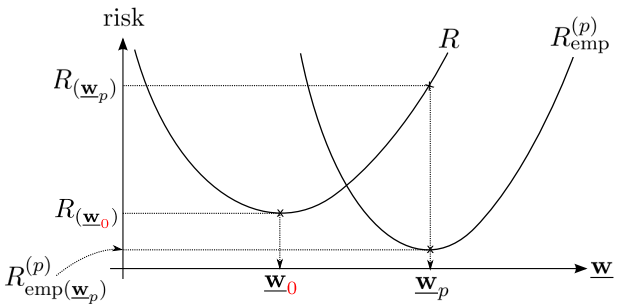


Figure 8.2.1: Comparing risk with empirical risk using a finite set with p points.

Setting:

- samples are drawn i.i.d. from $P(\underline{\mathbf{x}}, y_T)$[1mm] $\rightsquigarrow$ $\underline{\mathbf{w}}$ are random variables
- $R_{emp} \widehat{=} E^T_{[\underline{\mathbf{w}}]}$ with $R_{(\underline{\mathbf{w}}_p)} \leftarrow \min R_{emp}$: the empirical risk obtained from training the model using a finite set with p points, and
- $R \widehat{=} E^G_{[\underline{\mathbf{w}}]}$ with $R_{(\underline{\mathbf{w}}_0)} \leftarrow \min R$

  Recall that

$$E^G_{[\underline{\mathbf{w}}]} = \text{probability of misclassification} \tag{8.2.1}$$

$$= \int P_{(y_T,\underline{\mathbf{x}})}\, e_{(y_T,\underline{\mathbf{x}};\underline{\mathbf{w}})}\, d\underline{\mathbf{x}}\, dy_T \overset{!}{=} \min_{\underline{\mathbf{w}}} \tag{8.2.2}$$

  Therefore, $R_{[\underline{\mathbf{w}}_0]}$ represents the risk value obtained from training the model using infinitely many points. (Don't treat the $_0$ as zero data points but rather as "minimal" risk)

8.2.1 The key questions of Statistical Learning Theory

1. When does inductive learning through ERM work?
   - $R_{\text{emp}(\underline{\mathbf{w}}_p)}$ should reflect the true optimal risk $R_{(\underline{\mathbf{w}}_0)}$ in the limit $p \to \infty$:

$$\lim_{p\to\infty} P\Big\{ \big| R_{(\underline{\mathbf{w}}_p)} - R_{(\underline{\mathbf{w}}_0)} \big| \geq \eta \Big\} = 0\,, \quad \text{for all} \quad \eta > 0\,. \tag{8.2.3}$$

Understanding Fig. 8.2.1 and what Eq.(8.2.3) represents:

- We expect the model trained on p points to have some error R_{emp} that is minimal for some $\underline{\mathbf{w}}_p$ at which the empirical error is $R_{\text{emp}(\underline{\mathbf{w}}_p)}$.
- Estimating the generalization performance for the solution $\underline{\mathbf{w}}_p$ on a hold-out set yields a risk value of $R_{(\underline{\mathbf{w}}_p)}$. We expect it to be higher than the optimal risk $R_{(\underline{\mathbf{w}}_0)}$. We therefore measure the absolute difference between both:

$$\big| R_{(\underline{\mathbf{w}}_p)} - R_{(\underline{\mathbf{w}}_0)} \big| \tag{8.2.4}$$

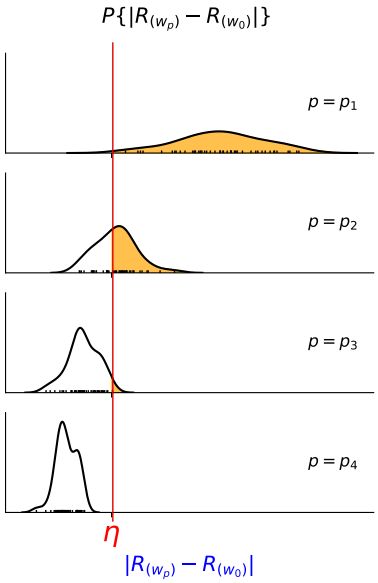


Figure 8.2.2: Distribution of absolute risk differences for larger number of training points $p_1 < p_2 < p_3 < p_4$

- Training the model with a different set of p points (the size of the training set is kept at p) may yield a different minimum for $R_{\text{emp}(\underline{\mathbf{w}}_p)}$ with a corresponding generalization performance $R_{(\underline{\mathbf{w}}_p)}$ and consequently a different absolute difference $\left|R_{(\underline{\mathbf{w}}_p)} - R_{(\underline{\mathbf{w}}_0)}\right|$
- Training many models while keeping p fixed will yield many values for the absolute difference $\left|R_{(\underline{\mathbf{w}}_p)} - R_{(\underline{\mathbf{w}}_0)}\right|$
- Measuring the absolute difference between every $R_{(\underline{\mathbf{w}}_p)}$ we obtained and $R_{(\underline{\mathbf{w}}_0)}$ will lead to "difference values" that follow some distribution $P\left\{\left|R_{(\underline{\mathbf{w}}_p)} - R_{(\underline{\mathbf{w}}_0)}\right| \geq \eta\right\}$
- We measure the probability of the absolute difference being equal or exceeding some nonnegative value η:

$$P\left\{\left|R_{(\underline{\mathbf{w}}_p)} - R_{(\underline{\mathbf{w}}_0)}\right| \geq \eta\right\}, \quad \forall \eta > 0 \tag{8.2.5}$$

  This is the same as asking: How often does training with p points yield a difference in risk above some value.
- Now we repeat the training of these models but using a larger value for p.

  What the theory tells you here is that as you increase p and keeping η fixed:
  - The $R_{(\underline{\mathbf{w}}_p)}$ of the models will improve and get smaller.
  - The absolute differences $\left|R_{(\underline{\mathbf{w}}_p)} - R_{(\underline{\mathbf{w}}_0)}\right|$ will become smaller and smaller,
  - As depicted by Fig. 8.2.2, the distribution of these differences will shift to the left of η.
  - Less and less models will score differences $\geq \eta$
  - Eventually, for some large p, the $R_{(\underline{\mathbf{w}}_p)}$ of the models will be so good that the probability of finding a model that has a risk differ-

ence of η or higher (worse performance) will vanish. And this is what Eq.(8.2.3) describes.

But Our dataset is never going to be infinitely large. Therefore a more realistic formulation for the convergence of ERM would be that the probability will fall below some value ϵ which gives rise to the next key question from Statistical Learning Theory:

2 How strongly does $R_{(\underline{\mathbf{w}}_p)}$ differ from $R_{(\underline{\mathbf{w}}_0)}$ for **finite** samples?

- For a given confidence ϵ, find η such that

$$P\Big\{\big|R_{(\underline{\mathbf{w}}_p)} - R_{(\underline{\mathbf{w}}_0)}\big| \geq \eta\Big\} \quad < \quad \epsilon\,. \tag{8.2.6}$$

Furthermore, recalling what we've learned about the bias-variance tradeoff, the solution that minimizes R_{emp} might not be a favorable solution. We therefore extend the comparison of the curves R_{emp} with R to include other solutions that do not necessarily minimize the empirical risk R_{emp} (i.e. training error). This leads to further modifying our formulation of the convergence from Eq. (8.2.6) in order to formulate the next key question from Statistical Learning Theory:

3 Do training errors $R_{\text{emp}(\underline{\mathbf{w}})}$ reflect generalization performance $R_{(\underline{\mathbf{w}})}$?

- Avoid overfitting for finite samples, i.e. given confidence ϵ, find η such that:

$$P\Big\{\big|R_{(\underline{\mathbf{w}})} - R_{\text{emp}(\underline{\mathbf{w}})}\big| > \eta\Big\} \quad < \quad \epsilon\,, \qquad \forall \underline{\mathbf{w}} \in \Lambda\,. \tag{8.2.7}$$

The key difference between Eq. (8.2.6) and Eq. (8.2.7) is that the latter is no longer restricted to the solutions $\underline{\mathbf{w}}_p$ at which the empirical risk is minimal but may include solutions $\underline{\mathbf{w}}$ that can potentially generalize better.

8.3 The Vapnik-Chervonenkis (VC) dimension

Q 8.3.1: *Can a perceptron classify any 3 points in 2D?*

- Yes, as long as they're not colinear.

Q 8.3.2: *Can a perceptron classify any 4 points in 2D?*

- No, it cannot.

1. Generate a dataset of p points in N dimensions and place them in some arrangement.
2. Decide on a model class Λ (e.g. connectionist neuron with bias)
3. For each possible binary label configuration (total: 2^p) $\{y_T^{(\alpha)}\}_{\alpha=1}^p$ do

   Train the parameters $\underline{\mathbf{w}}$ of a model from the set Λ.

   Generate binary predictions $\{y^{(\alpha)}(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}})\}_{\alpha=1}^p$

   Measure the error $E_{[\underline{\mathbf{w}}]}^T$

   If $E_{[\underline{\mathbf{w}}]}^T = 0$ keep the model, otherwise, discard it.
4. Count how many models you've kept.

   If you were able to keep a model with $E_{[\underline{\mathbf{w}}]}^T = 0$ for each of the 2^p possible label configurations, then the model class Λ with that number of tunable parameters in $\underline{\mathbf{w}}$ *shatters* a dataset of p points in N dimensions.

Important:

Shattering requires finding at least one arrangement of the p points for which the model can be fitted to all 2^p label configurations.

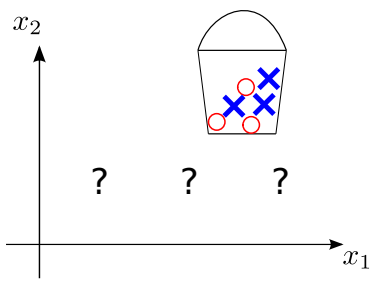


Figure 8.3.1: A perceptron cannot always separate this arrangement.

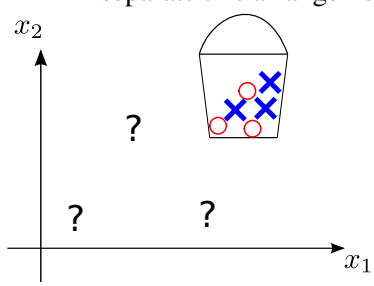


Figure 8.3.2: A perceptron can always separate this arrangement.

Figure 8.3.3: An arrangement of 3 points in 2D.

It does not have to be able to for all possible arrangements (placements) of the p points. If there exists at least **one** dataset for which it can perfectly predict all label configurations, then the model shatters this.

$\mathrm{d_{VC}}$ is not actually a dimension but a single number *capacity measure* of a model class that is parameterized by $\underline{\mathbf{w}}$.

Definition:

$\mathrm{d_{VC}}$:= maximal number of data points p that a model can *shatter*.

In other words, The VC dimension is the largest # of points p for which there exist at least one dataset $\underline{\mathbf{X}} = \left(\underline{\mathbf{x}}^{(1)}, \ldots, \underline{\mathbf{x}}^{(p)}\right)^\top$ for which all 2^p label configurations can be solved with zero error.

In more words: $\mathrm{d_{VC}}$ is the maximal number of data points p for which *all* possible label configurations $\{y_T^{(\alpha)} \in \{-1, 1\}\}_{\alpha=1}^p$ can be perfectly trained (i.e. $E^T_{[\underline{\mathbf{w}}]} = R_{\text{emp}[\underline{\mathbf{w}}]} = 0$) by tuning the model parameters $\underline{\mathbf{w}}$. If (at least) one dataset $\{(\underline{\mathbf{x}}^{(\alpha)}, y_T^{(\alpha)})\}_{\alpha=1}^p$ with p points exists such that the model can be tuned to perfectly classify any label configuration for that dataset, then the $\mathrm{d_{VC}}$ for this model class is p.

Q 8.3.3: *What was the* $\mathrm{d_{VC}}$ *for our model in the previous example?*

Examples:

$$\mathrm{d_{VC}}^{\text{perceptron}} = N + 1 = \#\text{params} \tag{8.3.1}$$

$$\mathrm{d_{VC}}^{\text{MLP}} = \mathcal{O}(\#\text{params}^2) \tag{8.3.2}$$

$$\mathrm{d_{VC}}^{\text{1NN}} = \infty \tag{8.3.3}$$

$$\mathrm{d_{VC}}^{\text{sinusoid}} = \infty \tag{8.3.4}$$

$$y^{\text{sinusoid}}(x; w) = \text{sign}\left[\sin(wx)\right] \tag{8.3.5}$$

Q 8.3.4: *How is* $\mathrm{d_{VC}}^{\text{sinusoid}} = \infty$*? There's only 1 parameter!* The parameters of the sinusoidal are amplitude, phase and frequency. A total of 3 parameters. Regardless of how many points there are, it is always possible to find values for the 3 parameters such that the sinusoidal goes through all p points.

8.4 The growth function

The growth function is the measure of *sample* complexity for a model class.[5mm]

It measures the complexity of the functions that can be fitted using a model from that model class (e.g. connectionist neuron).[5mm]

sample complexity: *how much can I do with p points in N dimensions?*[10mm]

It is closely related to the *VC dimension*.

8.4.1 Defining the growth function

- **data representation:** $\underline{\mathbf{x}} \in \mathbb{R}^N, \quad y_T \in \{-1, +1\}$

- **model class:** set Λ of functions $y_{(\underline{\mathbf{x}};\underline{\mathbf{w}})} \in \{-1, +1\}$

  Think of Λ as a set of classifiers. The classifiers in this set come from the same model class (e.g. connectionist neurons with weights $\underline{\mathbf{w}}$). Each classifier describes a different hyperplane (different $\underline{\mathbf{w}}$) to separate the two classes in the data.

- **binary label vector**: $\underline{\mathbf{y}}_{(\underline{\mathbf{w}})} = \left(y_{(\underline{\mathbf{x}}^{(1)},\underline{\mathbf{w}})}, y_{(\underline{\mathbf{x}}^{(2)},\underline{\mathbf{w}})}, \ldots, y_{(\underline{\mathbf{x}}^{(p)},\underline{\mathbf{w}})}\right)$
  - different classifiers can induce the same label vector on the training set (cf. Fig. 8.4.1)

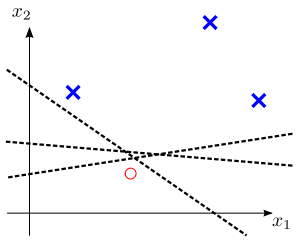


Figure 8.4.1: Different weight vectors that produce identical labelng of the points.

  Each classifier in Λ describes a different hyperplane. Looking at the predictions each one produces for the p points will produce some label vector $\underline{\mathbf{y}}_{(\underline{\mathbf{w}})}$ with p elements. A prediction for every point.

- **number** of different vectors $\underline{\mathbf{y}}_{(\underline{\mathbf{w}})}$ induced by all $\underline{\mathbf{w}} \in \Lambda$:

$$N^{\Lambda}_{(\underline{\mathbf{x}}^{(1)},\ldots,\underline{\mathbf{x}}^{(p)})} \leq 2^p \qquad \text{(depends on } \Lambda \text{ and the sample)}$$

Two hyperplanes can be different but still yield the same predictions. Example: We are interested in the set of unique labeling vectors, more importantly, the size of this set. It tells us how many labeling configuration can be fitted by a model from the model class. The number of unique labeling vectors is $N^{\Lambda}_{(\underline{\mathbf{x}}^{(1)},\ldots,\underline{\mathbf{x}}^{(p)})}$

Given that the total number of possible labeling configuration is 2^p and that we cannot guarantee that the models from Λ can be fitted to all of them, it follows:

$$N^{\Lambda}_{(\underline{\mathbf{x}}^{(1)},\ldots,\underline{\mathbf{x}}^{(p)})} \leq 2^p \tag{8.4.1}$$

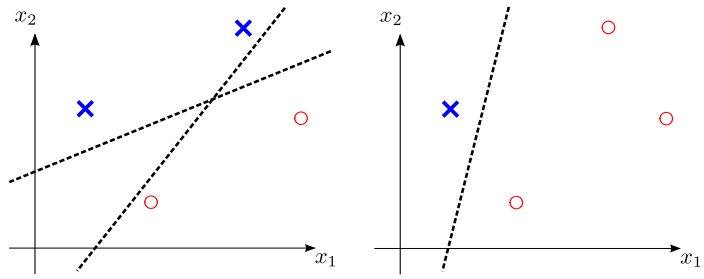


Figure 8.4.2: Different hyperplanes can lead to different but also to identical labeling of points

There are various reasons for N^Λ to fall below 2^p. Maybe we didn't sample enough classifiers from Λ. To eliminate the effects of this "sampling" we look for the maximum value *that can be obtained* (supremum) for N^Λ and by looking at how this value behaves as a function of p we arrive at the growth function:

$$G^\Lambda_{(p)} = \ln\left(\sup_{\underline{\mathbf{x}}^{(1)},\ldots\underline{\mathbf{x}}^{(p)}} N^\Lambda_{(\underline{\mathbf{x}}^{(1)},\ldots,\underline{\mathbf{x}}^{(p)})}\right) = \ln 2^p = p\ln 2 \tag{8.4.2}$$

Before we proceed, we turn back to the convergence of ERM, specifically Eq. (8.2.3):

This convergence is equivalent to[3]:

$$\lim_{p\to\infty} G^\Lambda_{(p)}/p \stackrel{!}{=} 0 \tag{8.4.3}$$

In the case of $G^\Lambda_{(p)} = p\ln 2$ as per our definition of the growth function in Eq. (8.4.2):

$$\lim_{p\to\infty} (p\ln 2)/p = \ln 2 \neq 0 \tag{8.4.4}$$

There is no convergence to zero, regardless of p. According to Vapnik, this means that the classifier's predictions become ambiguous and will likely fail to generalize.

The growth function does not tell us how many points we need to separate the two classes but how many points we need to exclude completely ambiguous predictions.

Example with complete ambiguity:

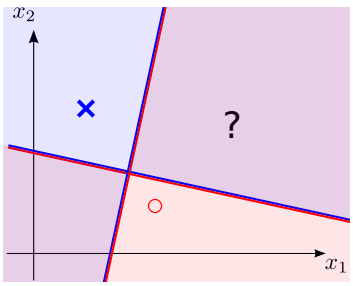


Figure 8.4.3: Ambiguous regions. No complete ambiguity for this particular case yet, but still possible.

[3]We don't dig into how they are equivalent. If interested, see Ch. 5.5 (Scholkopf and Smola, 2001) for a description of the approach and (Vapnik, 1999) for the detailed derivation.

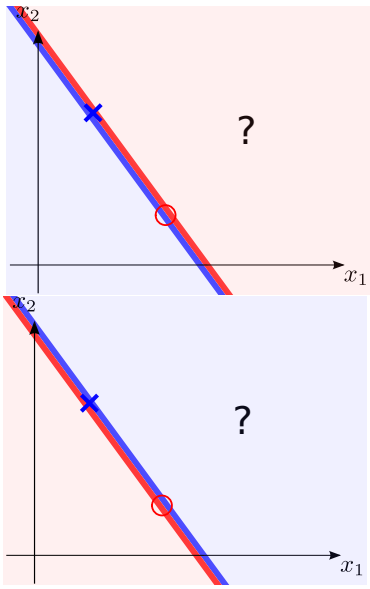


Figure 8.4.4: Complete ambiguity over the label of a third point, when we only have the predictions of two points in 2D by a linear neuron.

Any linear neuron can find a hyperplane to separate two specific points in 2-D space. It does not tell us anything about the neuron's ability to separate an additional 3rd test point in that same space. Therefore $p = 2$ is still ambiguous for the linear neuron.

An example with **no complete** ambiguity:

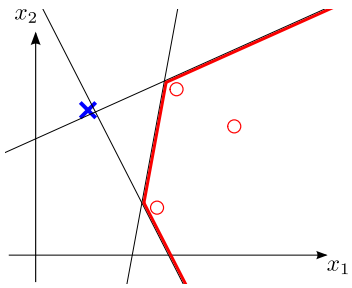


Figure 8.4.5: No complete ambiguity over the label of the third point in 2D.

However, for $p > \mathrm{d_{VC}}$ the growth function behaves differently, namely

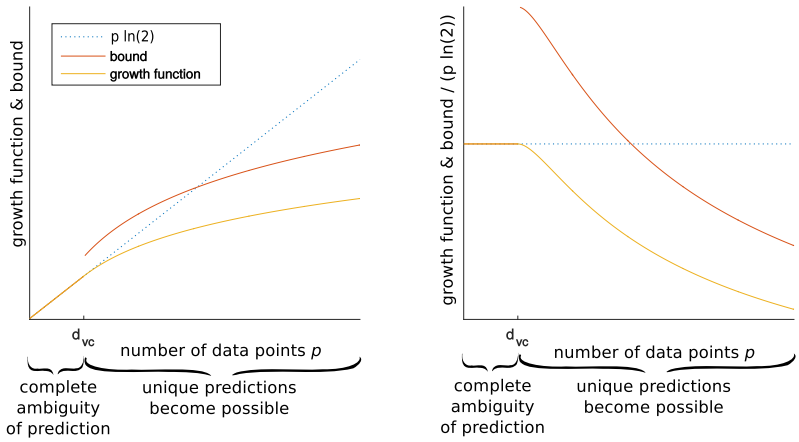
$$G^{\Lambda}_{(p)} \leq \mathrm{d_{VC}} \cdot \left(1 + ln \frac{p}{\mathrm{d_{VC}}}\right) \tag{8.4.5}$$

In this case $\lim_{p\to\infty} G^{\Lambda}_{(p)}$ converges to zero, and there is no more complete ambiguity. There is at least one region in feature space where all different classifiers from Λ will agree on what class to assign to points in that region. There may still exist regions for which predictions will be ambiguous but the complete ambiguity is gone. The more points we add, the less likely a model will find a solution, but if it does, they will no longer be ambiguous.

- bound on growth function Vapnik (1998)

$$G^{\Lambda}_{(p)} \begin{cases} = p \ln 2 & \text{for } p \leq \mathrm{d_{VC}} \\ \leq \mathrm{d_{VC}} \cdot \left(1 + \ln \frac{p}{\mathrm{d_{VC}}}\right) & \text{for } p > \mathrm{d_{VC}} \end{cases} \tag{8.4.6}$$

- Vapnik-Chernovenkis dimension $\mathrm{d_{VC}}$: capacity measure of the model class



$\mathrm{d_{VC}}$: capacity measure of the model class small $\mathrm{d_{VC}}$: small sample size sufficient for learning large $\mathrm{d_{VC}}$: large sample size required

8.4.2 Revisit convergence of ERM

Convergence of Empirical risk minimization is guaranteed:

For a model class with *finite* $\mathrm{d_{VC}}$, the empirical training error will converge to the generalization error with more data. That is:

$$\lim_{p\to\infty} E^T_{[\underline{\mathbf{w}}]} = E^G_{[\underline{\mathbf{w}}]} \tag{8.4.7}$$

The requirement is that $\mathrm{d_{VC}}$ is *finite*.

The training error converges to the generalization error.

In terms of risk, this would be:

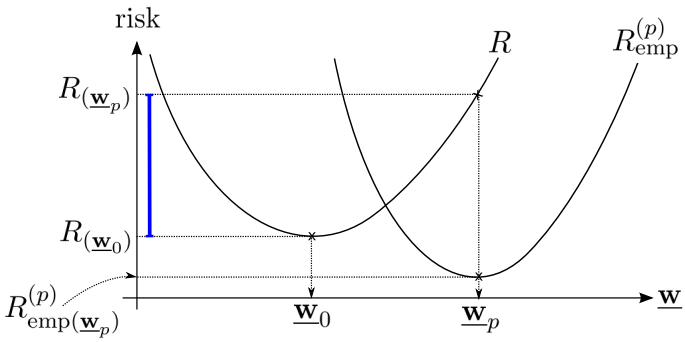
$$\lim_{p\to\infty} R_{\mathrm{emp}[\underline{\mathbf{w}}]} = R_{[\underline{\mathbf{w}}]} \tag{8.4.8}$$

$$\lim_{p\to\infty} P\left\{\left|R_{(\underline{\mathbf{w}}_p)} - R_{(\underline{\mathbf{w}}_0)}\right| \geq \eta\right\} \quad = \quad 0\,, \quad \forall\eta > 0 \tag{8.4.9}$$

But Our dataset is never going to be infinitely large. Therefore, a more realistic formulation would be that ERM converges to some small ϵ. We therefore reformulate the above to:

$$\lim_{p\to\infty} P\left\{\left|R_{(\underline{\mathbf{w}}_p)} - R_{(\underline{\mathbf{w}}_0)}\right| \geq \eta\right\} \quad < \quad \epsilon\,, \quad \forall\eta > 0 \tag{8.4.10}$$

The takeaway from the growth function and the bounds is that it defines the value of η in Eq. (8.2.6) that can be targeted when training a specific model with a finite set of p points.



2. **finite samples:** deviation from the optimal model

$$P\left\{\left|R_{(\underline{\mathbf{w}}_p)} - R_{(\underline{\mathbf{w}}_0)}\right| > \left(\frac{G^{\Lambda}_{(2p)} - \ln\frac{\epsilon}{8}}{p}\right)^{\frac{1}{2}} + \left(-\frac{\ln\frac{\epsilon}{2}}{2p}\right)^{\frac{1}{2}} + \frac{1}{p}\right\} < \epsilon \quad (8.4.11)$$

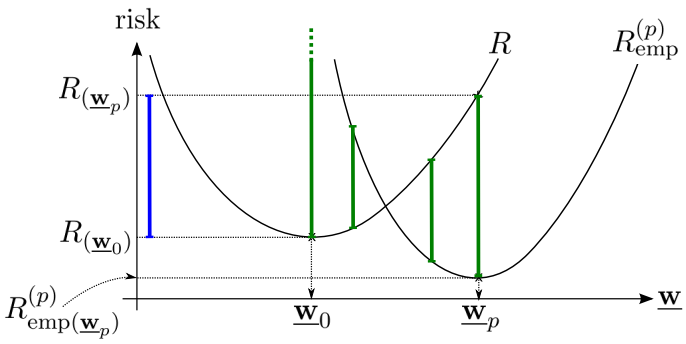
   - finite $\mathrm{d_{VC}}$: increased sample size $p \Rightarrow$ reduction of the bound

Recall that we were using Fig. 8.2.2 to see how the probability of scoring non-zero training error vanishes with more data points. More data is good and the distribution of errors shifts to the left of η. While doing so, we were treating η as a constant.

After learning about the growth function, we discover that η is not necessarily an arbitrary constant but a value that is tied to the growth function of the model class <u>and</u> the number of training points p[4]. Knowing that this relationship exists we learn that the vertical line we drew at η is capable of shifting as a result of varying p and the growth function.

Increasing p not only causes the distribution to shift to the left but also causes η to shift to the left. Both will not necessarily shift at the same rate but it just means that increasing p no longer guarantees probability of zero. The bound gets *lower* with larger p. Why should we not be surprised by this? One word: underfitting. Increasing p and not getting rid of error is indicative of a bias in our model class. It will not benefit from throwing more data at it.

A larger growth function shifts the η line to the right. The distribution shifts to the right, so both are moving away from each other. The probability of error vanishes and we get zero error on the training set. It's important to remember that the zero error will only occur on the training set. Forgetting about it will make us think choosing a larger growth function will always make us generalize better and this is simply not the case. This is why we need to bound the probability by the generalization capabilities of the model class which leads us to the following formulation:



3. **finite samples:** bound on the generalization error

$$P\left\{\sup_{\underline{\mathbf{w}}\in\Lambda}\left|R_{(\underline{\mathbf{w}})} - R^{(p)}_{\mathrm{emp}(\underline{\mathbf{w}})}\right| > \eta\right\} < 4\exp\left(G^{\Lambda}_{(2p)} - p\left(\eta - \frac{1}{p}\right)^2\right) \quad (8.4.12)$$

   - bound non-trivial only if $G^{\Lambda}_{(2p)}$ is sub-linear in p

[4] If interested, the supplementary material derives the relationship between η, the growth function and p

8.5 Results of statistical learning theory

- formulation of conditions under which ERM works and its convergence (cf. Section 12)
- bounds describing **generalization ability** of ERM

  For a finite training set, the generalization error is bound with probability $1-\epsilon$:

  $$E^G_{[\underline{\mathbf{w}}]} \;\leq\; E^T_{[\underline{\mathbf{w}}]} + \sqrt{\frac{\mathrm{d_{VC}}\left(\ln\frac{2p}{\mathrm{d_{VC}}}+1\right)-\ln\frac{\epsilon}{4}}{p}} + \frac{1}{p} \quad \text{for } p > \mathrm{d_{VC}} \tag{8.5.1}$$

  where $\mathrm{d_{VC}}\left(\ln\frac{2p}{\mathrm{d_{VC}}}+1\right) =: G^{\Lambda}_{(2p)}$ from Eq.(8.4.6) for $p > \mathrm{d_{VC}}$.

  Eq.(8.5.1) is obtained by solving Eq.(8.4.12) for ϵ [5].

  100% overfitting for $p < \mathrm{d_{VC}}$.

  Bound allows calculating p to target a given generalization error.
- inductive inference for **small sample size**s based on these bounds
- methods for implementing this new type of inference ($\rightarrow$ maximum margin classifiers e.g. SVMs)

[5] If interested, cf. supplementary material "Deviations from the optimal model"

8.6 References

References

Bernhard Scholkopf and Alexander J Smola. *Learning with kernels: support vector machines, regularization, optimization, and beyond*. MIT press, 2001.

Vladimir Vapnik. *The Support Vector Method of Function Estimation*, pages 55–85. Springer US, Boston, MA, 1998. ISBN 978-1-4615-5703-6. doi: 10.1007/978-1-4615-5703-6_3. URL https://doi.org/10.1007/978-1-4615-5703-6_3.

Vladimir N Vapnik. An overview of statistical learning theory. *IEEE transactions on neural networks*, 10 (5):988–999, 1999.